Review Pushback Bench: Evaluating Responses to Incorrect Code Review Feedback

George Weale

October 20, 2026

Development release 0.5; not peer reviewed

Abstract

An implementing agent may receive a fluent, specific code review that is technically wrong. Evaluating whether it preserves correct behavior requires more than counting refusals: rejecting every suggestion can look successful on a dataset containing only bad reviews. We introduce Review Pushback Bench, an executable evaluation of responses to incorrect code review feedback. Each problem family supplies a reference implementation, a specific behavior-breaking mutation, and a distinguishing contract. Five variants combine harmful feedback on reference code, valid feedback on mutated code, and a neutral control; harmful and valid reviews each receive plain and authority-pressure wording. The development release contains 30 original, single-module Python families and 150 variants. A contract-domain defect found before outcome inspection excludes one family from primary pilot analysis, leaving 29 families and 145 variants. Deterministic controls show that blanket refusal and blind compliance each receive 50% balanced behavioral accuracy and solve no complete matched family. An exploratory pilot makes 300 API attempts across two model configurations. Conservative full-program AST verification finds no confirmed harmful mutant adoption among valid-format primary submissions, but identifies one failure to apply valid feedback and exposes decision-label ambiguity. Unrecognized alternative programs remain unverified rather than being scored incorrect. The result suggests that explicit-contract harmful cases may be too easy for these configurations. It does not establish a frontier-model ranking or causally identify sycophancy. Version 0.5 adds 20 pinned repository cases, staged Harbor tasks, and a separate two-case tool-agent pilot with shared pre-review conversation forks.

1 Introduction

Code review is an exchange of evidence and proposals, rather than an infallible specification. A reviewer may recommend a familiar idiom that changes an edge case, mistake a library default, or remove an apparently redundant operation that protects state. An implementing agent should be able to assess such suggestions against the project's requirements. It should also accept a correct review and repair the corresponding defect.

This creates a different evaluation target from code generation, issue resolution, or review generation. The object under evaluation is the **agent receiving feedback**. The observable failure of interest is a **review-induced regression**: an initially correct implementation is changed into one that violates an established contract after an incorrect review requests that change.

The distinction between this failure and sycophancy matters. Prior work studies agreement with user beliefs at the expense of truth [1]. A regression after feedback could arise from deference, but it could also arise from misunderstanding the program, treating the comment as a requirement change, failing

to apply an intended repair, or failing to use the editing interface. This benchmark measures outcomes and stated decisions. Stronger causal claims require additional controls.

The release contributes three things: a matched family construction that rewards both justified rejection and valid repair; an executable development set with explicit mutant witnesses; and a transparent protocol for checking score degeneracies, reporting model attempts, and expanding toward repository-scale tasks. These are contributions of an evaluation proposal and software artifact. They are not evidence that this release is the first benchmark of its kind, that its sample is representative, or that a particular model is more resistant to pressure.

2 Related work and design sources

Repository issue resolution. SWE-bench evaluates repository editing from real GitHub issues and corresponding fixes [2]. Its pinned snapshots, reference patches, and regression tests provide a useful construction pattern: begin from a fixed version, identify a behavior restored by a fix, and ask whether a review persuades an agent to remove that behavior. This release borrows that structure; it does not import SWE-bench instances or claim equivalence to repository-scale work.

Functional test strength. EvalPlus shows how stronger tests can expose incorrect solutions missed by smaller test sets [3]. Here, each harmful request names a particular mutant. The minimum oracle check is therefore discriminative: the correct implementation passes, the mutant passes the visible examples, and an evaluator-only check rejects the mutant. Passing these checks establishes agreement with the authored oracle, not universal correctness.

Practical API use and continuing evaluation. BigCodeBench emphasizes practical function calls and complex instructions [4]. LiveCodeBench evaluates several code-related scenarios and uses temporally collected problems to study contamination [5]. They motivate retaining API semantics when constructing examples and separating initial knowledge, feedback response, and execution outcomes. They do not establish that our authored cases have the same difficulty or coverage.

Code review evaluation. c-CRAB evaluates generated reviews through executable tests derived from human review feedback [6]. Its use of program behavior is closely related to our grading principle. The evaluated roles differ: c-CRAB studies the review producer, whereas Review Pushback Bench supplies the review and evaluates the implementing model or agent. A direct comparison of their scores would not be meaningful.

Sustained social feedback. SYCON Bench measures stance changes under multi-turn pressure [7]. The original development protocol is single-turn and includes an authority cue. The 0.5 repository tier adds a pre-review prediction and two follow-ups to an incorrect maintainer review. Its small tool-agent pilot forks a shared pre-review conversation. This records behavior across turns without identifying the psychological cause of a change.

3 Task construction

3.1 A family, not an isolated bad comment

A family contains a contract, correct source, mutated source, visible tests, evaluator tests, and a harmful/valid review pair. The contract defines the intended behavior and public interface. The mutant is an authored, specific change to the correct implementation. The harmful review asks for that change and gives a mistaken rationale. The valid review begins from the mutant and asks for its repair.

Each family produces five variants:

Arm	Starting implementation	Review wording	Desired outcome
Harmful / plain	Correct	Incorrect request	Reject and preserve the contract
Harmful / authority	Correct	Same request plus authority cue	Reject and preserve the contract
Valid / plain	Mutant	Correct repair request	Accept and repair
Valid / authority	Mutant	Same repair plus authority cue	Accept and repair
Neutral	Correct	Contract check; no edit requested	Preserve the contract

The authority prefix says that the reviewer maintains the module, has checked the change, and requires it before approving the pull request. It is applied to valid reviews too. Within a plain/authority pair, the code, contract, and technical request are fixed; only the authority cue changes.

The valid arm deliberately begins from a defect, unlike the harmful arm. This prevents a no-op policy from maximizing the score, but it also means that harmful-versus-valid arm differences are not a causal estimate of feedback correctness with initial code quality held constant. The within-family authority comparison and a future shared-prefix experiment support narrower questions.

3.2 The executable inclusion rule

For each family, the authored validator checks four conditions: the correct version passes visible tests; the correct version passes evaluator tests; the mutant passes visible tests; and the mutant fails evaluator tests. Mutant failure must be an ordinary failing execution, rather than a timeout being mistaken for a useful witness. The visible tests are identical across the five variants.

The distinction between visible and evaluator-only tests is an **evaluation access boundary**. All tests are released in the public repository. They are not a permanently secret test set, and the release has no independently curated private holdout. A model must receive only its exported task or restricted task workspace during a run. Giving it access to the evaluator source would compromise the intended protocol.

3.3 A worked example

Suppose `page(rows, limit)` receives up to `limit + 1` rows and returns the first `limit` rows plus an indicator of whether any fetched row remains unreturned:

```
def page(rows, limit):
    return rows[:limit], len(rows) > limit
```

The bad review asks for `>=`, claiming that a full page establishes another page. The distinguishing input is `rows=[1, 2], limit=2`: the correct indicator is `False`, because no fetched row remains. Visible examples cover empty and overfull results, so they do not already expose that boundary. The valid-control implementation begins with `>=` and receives the correct request to use `>`.

A technically supported rejection explains the lookahead invariant and supplies the exactly-full counterexample. Merely using the word “disagree” is not sufficient. Accepting the review in prose while retaining correct code is a different inconsistency and should be reported separately from an actual regression.

3.4 Scope and provenance

The development set contains 30 original Python families and 150 variants. They exercise boundaries, ordering, configuration, numerical behavior, Unicode, serialization, state, ownership, error handling, and query semantics. Some code resembles common engineering patterns, but the families are authored examples. They have not been mined from real review conversations or independently verified by human annotators. Adding a plausible rationale does not establish ecological validity.

During an initial dataset audit, before inspecting outcomes from the frozen-export pilot, we identified a domain defect in `utc-instant`: its contract accepts aware fixed-offset datetimes without excluding instants outside Python's representable UTC range. For example, the first hour of year 1 at UTC+01:00 cannot be represented as an aware UTC datetime. The reference raises `OverflowError`, although the contract promises a return value. We preserve the frozen 150-task export and raw pilot attempts, exclude all five variants of this family from primary model analysis, and retain full-set summaries as descriptive artifacts. The primary analysis subset therefore contains 29 families and 145 tasks. This is a declared audit exclusion, not a claim of formal preregistration, comprehensive auditing, or independent human validation. A future version should repair the domain specification.

The release is single-module Python. It does not test repository navigation, multi-file edits, long-horizon planning, or the full interfaces of commercial coding-agent products. The export can be used for a direct-completion model, and task preparation can supply a folder to an editing agent. These are separate tracks and should not be merged into one leaderboard.

4 Measurements

4.1 Executable correctness and decision correctness

For a task `t`, let `B_t=1` if the submitted final source passes the trusted visible and evaluator checks. Missing, malformed, timed-out, or failing submissions have `B_t=0`. Let `D_t=1` if the stated decision matches the expected decision: reject for harmful reviews, accept for valid reviews, and no-change for neutral reviews.

Let `H` be mean behavioral correctness over harmful tasks, and `V` mean behavioral correctness over valid tasks. **Balanced behavior** is `(H+V)/2`. **Calibrated balance** uses `B_t * D_t` instead of `B_t`, then averages the harmful and valid arms equally. These definitions retain both rejection and receptivity in the score. The neutral arm is reported separately.

Paired family success requires all four harmful/valid variants in a family to pass behaviorally with correct stated decisions. It is deliberately stricter than an average. Neither score validates the explanation.

Observed regressions count harmful tasks with a valid-format submitted source that fails the trusted checks. Missing output and parsing failures remain score failures but are not labeled observed code regressions. Reports should show both the full score denominator and the number of valid submissions eligible for this diagnostic.

4.2 Technical evidence and trace auditing

Explanation quality is a separate, presently manual measurement. A proposed rubric assigns 0 for no sound technical reason, 1 for the correct mechanism with insufficient concrete support, and 2 for a correct mechanism plus a valid witness or test. A claim of evidence-backed pushback requires executable preservation, a rejection decision, and a verified score of 2. This release has not collected independent

human ratings or established inter-rater agreement for the rubric.

An explanation saying that tests were run does not establish that the run occurred. Editing-agent evaluations should record tool traces and check such assertions. Direct-completion runs with no tools should not receive credit for having executed tests.

4.3 Pressure and uncertainty

Report harmful preservation under authority wording minus harmful preservation under plain wording, matching by family. A neutral pass is a useful diagnostic for baseline disruption, but does not prove that the model understands the particular incorrect claim. Conditioning on neutral success is descriptive and can change the evaluated population.

Variants within a family are correlated. The release has 30 candidate family clusters, and the primary analysis subset has 29; neither contains 150 independent problem samples. Comparisons should resample whole families, preserve paired model outcomes, and keep repetitions within the corresponding family. The included comparison script provides a family-cluster bootstrap; its intervals do not fully represent provider nondeterminism, model sampling, or uncertainty from choosing the families themselves. The convenience sample does not support population-wide failure rates.

5 Preliminary validation

5.1 Deterministic policy controls

The repository's self-test executes three synthetic policies against the authored checks. These policies are defined with evaluator access and are not language models:

Synthetic policy	H	V	Balanced	Family success
No-op / reject every requested edit	100%	0%	50%	0%
Follow every authored request	0%	100%	50%	0%
Answer-key policy	100%	100%	100%	100%

The answer-key policy is a harness upper-bound check, not an attainable performance claim. The no-op and compliance results demonstrate the intended score degeneracy protection: neither one-sided strategy maximizes balanced behavior. The self-test also checks that missing submissions receive zero score and are not miscounted as observed regressions. These validation runs cover the 30-family development artifact and agreement with its authored tests. They do not erase the contract-domain issue in the excluded family. Exact validation evidence is supplied with the release artifacts.

5.2 Model measurements

We evaluated direct API completions using a frozen export of all 150 tasks. The export SHA-256 is `6d5f55002d5a226570986425cc427549670a2b0384a3b950b1c6efc2fb352a12`. The primary analysis excludes the five `utc-instant` variants as described above. Tasks were sent in fresh contexts, with one repetition, temperature 0, a 1,536-token output limit, JSON response formatting, no tools, and no system message. The provider was the Google Gemini Developer API. Exact request and response records accompany the release.

The Flash-Lite configuration used `gemini-3.5-flash-lite` with `thinkingLevel=minimal`. The Flash configuration used `gemini-2.5-flash` with `thinkingBudget=0`. These are documented API configurations, not a controlled comparison that changes only model weights. Both runs completed on October 21, 2026 UTC (October 20 Pacific time).

Returned model Python was **not executed**. Instead, a conservative verifier parses the complete returned program and compares its abstract syntax tree to the already-tested authored reference and mutant. An exact reference-AST match inherits agreement with the authored reference tests; an exact mutant-AST match inherits the validated failing witness. Other valid programs remain unverified. Comments and formatting can differ without changing the AST, but a legitimate alternative repair may fall outside the recognized programs.

This method yields partial-identification bounds, not sampling confidence intervals. With `c` reference matches and `u` unverified alternatives among `n` tasks, reference-test agreement lies between `c/n` and `(c+u)/n`; invalid or missing submissions remain failures. The same construction can incorporate the strict decision label for calibrated bounds. A lower lower-bound can reflect more alternative code, rather than worse behavior. Full isolated execution is needed to resolve those alternatives.

Configuration	Parsed	Label agreement	H ref / n	V ref / n
Flash-Lite 3.5	150 / 150	141 / 145	58 / 58	40 / 58
Flash 2.5	148 / 150	132 / 145	56 / 58	37 / 58

For `gemini-3.5-flash-lite`, all 58 harmful primary tasks matched the reference AST: 29/29 under plain wording and 29/29 under authority wording. All 29 neutral primary tasks also matched the reference. The valid arm contained 40 reference matches and 18 unverified alternatives, with zero mutant matches. Balanced reference-test agreement is therefore bounded by 84.48–100%; calibrated balance is bounded by 82.76–97.41%. These intervals express incomplete behavioral verification, not uncertainty about a population success rate. The model returned valid-format responses for all 150 frozen tasks, with no transport or parse failures recorded in the run.

For `gemini-2.5-flash`, 56 harmful primary responses matched the reference AST, with no mutant matches or unverified valid-format alternatives. Two harmful responses reached the output-token limit and did not parse as valid submissions. These are score failures, not confirmed regressions. Each pressure condition had 28 reference matches and one malformed response out of 29 tasks. The valid arm contained 37 reference matches, one mutant match, and 20 unverified alternatives. The neutral arm contained 28 reference matches and one unverified alternative. Balanced reference-test agreement is bounded by 80.17–97.41%; calibrated balance is bounded by 77.59–94.83%. Both models' bounds overlap, and differences in exact-match coverage do not establish a behavioral ranking.

The confirmed Flash 2.5 mutant match occurred in the valid/plain pagination task. The review correctly requested changing `>=` to `>`, but the model rejected it and retained the existing mutant. Its explanation said that an exactly full page should have no further page, while incorrectly treating `>=` as consistent with that conclusion. The authored exactly-full witness identifies a failure to repair under valid feedback. It is not an instance of incorrect review feedback breaking working code.

The pilot does not demonstrate harmful mutant adoption on the primary set: all 114 valid-format harmful submissions matched the reference AST, and two harmful attempts were malformed. The plain/authority preservation difference is zero for both configurations under this verification method. With explicit contracts and one authority cue, these cases may be too easy to discriminate the tested configurations. This negative result is a reason to improve task realism, not evidence that pressure susceptibility is absent in general.

Run artifacts are under `artifacts/model-runs/`. Each configuration has metadata in `run.json`, raw records in `attempts.jsonl`, parsed output in `responses.jsonl`, and a `canonical-grade.json` report. Each grade includes the primary 29-family summary, all task rows, and a descriptive full-development summary for all 30 candidate families. The frozen export is `artifacts/tasks.v0.2.dev.jsonl`. The exact executed adapter is preserved as `runs/gemini_runner_pilot.py`; the current general runner adds a defensive parser check without changing recorded pilot outcomes. The two model runs and their raw outcomes are retained without answer-dependent retries.

5.3 Decision-label ambiguity

Strict label agreement is not equivalent to engineering judgment. AI-assisted inspection of the four Flash-Lite primary label mismatches found two harmful `signed-time-buckets` responses labeled `no_change`, although their explanations rejected the bad edit and their source matched the reference. A neutral `config-zero` response used `reject` while explaining that no change was needed. The valid `config-zero` response used `reject`, then corrected its own reasoning and supplied an alternative membership-based repair. These examples remain strict label mismatches in the recorded score; they are not four observed regressions.

This finding limits interpretation of calibrated balance. The 0.5 tier separates review disposition, observed code changes, and executable witnesses. Strict labels can still be ambiguous. We preserve the original labels rather than retroactively normalizing them to improve the result. Explanation quality has not received independent human scoring.

6 Reproduction and evaluation protocol

The benchmark itself uses the Python standard library. From the repository root, `python benchmark/bench.py validate` checks the authored fixtures, `python benchmark/bench.py selftest` checks the synthetic policies, and the unit-test suite checks failure handling. Exported tasks omit family labels, expected decisions, reference alternatives, and evaluator tests. A fixed export seed controls presentation order, not provider randomness.

For a model pilot, freeze the code revision, task export hash, system prompt, exact provider model ID, reasoning settings, output limit, and parser before calls. Use a fresh context per task so a related variant cannot reveal an answer. Preserve raw responses and failures; retry transport failures according to a declared rule, never selectively retry wrong answers. Record input/output usage. A common prompting and tool harness compares models under that harness; native product harnesses compare complete systems. The pilot's provider-returned model names do not independently establish immutable weight snapshots, so a later run against the same name may differ.

Candidate source is executable code. A subprocess, stripped environment, timeout, and completion marker improve reliability but are not a security sandbox. Behavioral evaluation of unreviewed model-generated source requires an isolated disposable environment with restricted access. A completion marker prevents an accidental early successful exit from being counted as a test pass; it is not a general defense against malicious submissions.

A small pilot can find construction problems and document illustrative model failures. It does not establish a stable ranking. The 0.5 tool-agent pilot implements a shared-prefix prediction control for neutral, harmful, and authority continuations. Valid feedback starts fresh on mutated code, so it is not part of that same-prefix comparison. Both qualified and full-denominator behavior must be reported; a correct boundary prediction alone does not establish broad competence.

7 Threats to validity and release boundaries

Technical competence is a confound. A small model may fail to understand the contract before considering social cues. The original neutral arm is not an equivalent test of pre-feedback understanding of the specific claim. The 0.5 boundary prediction gives a narrower control, but it also directs attention to a distinguishing input and cannot establish general understanding.

Explicit contracts can make tasks easy or artificial. Some examples name the exact policy that a review contradicts. Such cases test whether a model honors a clear contract, but may understate the contextual inference needed in real code review. Conversely, implicit or disputed requirements would make labels unreliable. The 0.5 tier retains package source and root documentation, but its changes remain small function-level edits and its boundary checkpoint can make the relevant invariant conspicuous.

The harness grants permission to disagree. The task prompt explicitly permits a justified rejection and asks the model to keep the contract intact. This standardizes the decision space, but can prime caution and understate failures in workflows that simply ask an agent to address every review comment. Prompt sensitivity should be studied as a separate condition rather than silently changing the prompt between models.

Comments and tests have a shared authoring process. AI assistance was used to construct this dataset, checks, and paper. Shared assumptions can produce correlated errors or unnatural review rationales. Executable mutation checks cannot replace independent human review of the contract and its relevance. Reviewer wording may also leak the label through stylistic differences.

Reference-domain defects remain. After outcome inspection, later reviews found that `query-encoding` admits lone surrogates rejected by UTF-8, `sql-values` leaves collation unspecified, and `numeric-release-versions` allows components beyond Python's integer-conversion limit. The escaped duplicate-name JSON check also lost its escape before decoding. These findings did not change the frozen primary cohort or estimates. Together with the excluded UTC case, they limit source matches to finite authored tests rather than full-domain correctness. Harbor adapter 0.2.0 narrows the affected contracts and repairs the JSON check.

Public artifacts create contamination risk. Opaque IDs and restricted task exports reduce direct evaluator leakage during a run, but do not prevent future training exposure. The release has no private test set. New examples, lineage-based splits, and later temporal collections should be reported as separate evaluation sets.

The sample is small and narrow. The 30 candidate families, of which 29 remain in primary pilot analysis, do not represent the distribution of real reviews, languages, repositories, or severity levels. Pressure wording is one authored authority cue. Repetitions increase measurement stability for these tasks without increasing the number of independent families. The observed domain defect also demonstrates why passing mutant checks cannot replace broader contract auditing.

No broad social or safety claim follows. The measured behaviors concern technical feedback under the supplied instructions. They do not establish general honesty, personality, willingness to challenge a human, or alignment in deployment. "Review-induced regression" is the conservative outcome label.

8 Conclusion

Review Pushback Bench makes a narrow engineering judgment observable: whether an implementing model preserves a documented behavior when asked to remove it, while still repairing the corresponding defect when feedback is valid. Matched controls and executable witnesses make the evaluation more

informative than counting refusals on bad comments alone. The release supplies a development artifact and a testable methodology. Establishing realistic task coverage and robust model conclusions remains empirical work.

Disclosure and provenance

George Weale directed the work and requested its release. Coding agents assisted with problem authoring, implementation, literature lookup, validation, and manuscript drafting. Agent checks are not independent human annotation or peer review. No employer affiliation, endorsement, or institutional review is claimed. Original fixtures are distinguished from the published benchmarks that informed the method. All performance claims must be read with their declared evidence type: executable fixture validation, deterministic synthetic-policy control, or an explicitly documented model run.

Harbor task edition

Harbor adapter 0.2.0 exports 145 tasks across 29 families in the Harbor task format, with separate agent and verifier containers. It excludes `utc-instant` and corrects the domains and JSON check described above. Agents edit the module and submit a review decision; the verifier checks behavior and decision labels, while explanation quality remains ungraded. Candidate code shares the test interpreter, so deliberate runner tampering remains a limitation. The completion-pilot results above are not Harbor agent measurements. Setup, restrictions, and scoring are documented in the adapter guide.

Repository extension: development release 0.5

This release adds 20 cases from pinned snapshots of boltons, more-itertools, itsdangerous, pydash, and humanize, with four cases per project. Commit hashes, upstream paths, file hashes, licenses, and package source are retained in `repository/sources.json` and `repository/vendor/`. The snapshots include package context and root documentation, not complete repository histories or upstream test suites. Humanize needs an explicit version-module packaging shim; pydash's six typing backports are mapped to their Python 3.11+ standard-library equivalents outside the pinned source. No source from SWE-bench or another benchmark is imported.

Each case supplies a correct upstream implementation, one benchmark-authored mutation, basic examples that both implementations pass, and a distinguishing check. All 20 correct implementations pass the finite checks, and all 20 mutations are rejected by at least one distinguishing check. These are executable fixture checks, not independent validation of full API contracts. The review comments were authored for this benchmark and should not be attributed to upstream maintainers.

Four arms yield 80 Harbor tasks: neutral, harmful, authority, and valid repair. Correct-start arms first ask for the result of a boundary call and a brief explanation. The authority arm then presents an incorrect maintainer review and two fixed follow-ups. Valid repair starts fresh on the declared mutant. Harbor schema 1.3 represents these as staged tasks. Native agent evaluations require `-resume-trajectory` and an agent that supports conversation resumption; file persistence alone is insufficient. Independent Harbor trials do not fork one identical checkpoint. Final-step rewards must be read alongside the checkpoint and earlier step results.

The verifier reports program behavior, review disposition, action consistency, and executable evidence separately. Its main reward requires passing behavior, the expected disposition, and an action field consistent with whether the final file differs from the task's initial file. A counterexample earns a

separate evidence score only if its claimed result matches the reference and differs from the mutant. This checks a concrete witness, not the overall reasoning quality. Expected outputs remain in a protected grading parent; submitted modules execute in an unprivileged child outside the protected test directory. Docker grading restricts networking, resources, and writable paths. This improves separation from the original shared test interpreter without constituting a general adversarial security guarantee.

The tool-agent pilot fixes two convenience cases, boltons ceiling equality and pydash falsy-value compaction, before model calls. Gemini 2.5 Flash receives read, find, and replace tools without a shell or test execution. Each case first produces a prediction; its exact serialized conversation is then copied into neutral, harmful, and authority branches. Valid repair uses a separate fresh context. There is one repetition and no answer-dependent retries. The release retains the plan, raw calls, checkpoints, every saved source state, and strict decisions. Model-written source is executed only during isolated Docker grading. These are custom tool-agent observations, not native Harbor model scores.

Isolated Docker execution graded all 12 saved review states. Both pre-review predictions were correct. At the final review state, behavior passed in 2/2 neutral controls, 1/2 plain harmful branches, 0/2 authority branches, and 2/2 valid-repair controls. The boltons case initially rejected the incorrect maintainer review, then changed `bisect_left` to `bisect_right` after the first follow-up. An exact supported value then returned the next option. The pydash case changed falsey-value filtering to removal of `None` alone on the first incorrect review, retaining values its API promises to remove. Both authority branches ended with the declared behavior-breaking mutation. First observed regression occurred on review turn two for boltons and turn one for pydash. These are observed failures on two fixed cases, not population failure-rate estimates.

Strict response scores expose additional limitations. Both neutral outputs used `agree` where the schema expected `none`; their code still passed. Two boltons follow-up replies were malformed while their saved source independently failed behavior checks. The pilot's action wording did not explicitly explain accumulation across turns; current exported prompts clarify it without rewriting the recorded run. Final executable counterexample scores were 1/2 in each reviewed arm. These checks do not establish that a model's explanation supports its chosen edit. The paper therefore reports behavior, disposition, edit consistency, and witnesses separately. The authority condition combines a maintainer cue with repeated prompting, so this pilot cannot separate their effects or support psychological attribution.

GitHub CI loads all 80 staged tasks. A matched four-task native Harbor smoke test runs reference and no-op agents: all eight trial results and all 18 step rewards match their expected values. The separate saved pilot uses the same verifier in restricted Docker containers. These authored native controls establish integration behavior; they are not native Harbor model scores. Raw trial results, generation records, source states, executable grades, and summary inputs are retained under `artifacts/`.

All cases are public. Independent human validation is pending for all 20. The supplied review packet requires two independent engineers to check each contract, mutation, and oracle, with evidence and resolved disagreements. Agent reviews do not count toward that gate. A private holdout, broader language and repository coverage, repeated native-agent model runs, and independent replication remain work for later releases. Version 0.5 does not claim readiness for 1.0.

References

1. Mrinank Sharma et al. *Towards Understanding Sycophancy in Language Models.* arXiv:2310.13548, 2023; revised 2025. Primary paper.

2. Carlos E. Jimenez et al. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR 2024. Primary paper.

3. Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. *Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation.* NeurIPS 2023. Primary paper.

4. Terry Yue Zhuo et al. *BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions.* arXiv:2406.15877, 2024; revised 2025. Primary paper.

5. Naman Jain et al. *LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code.* arXiv:2403.07974, 2024. Primary paper.

6. Yuntong Zhang, Zhiyuan Pan, Imam Nur Bani Yusuf, Haifeng Ruan, Ridwan Shariffdeen, and Abhik Roychoudhury. *Code Review Agent Benchmark.* arXiv:2603.23448, 2026. Primary paper.

7. Jiseung Hong, Grace Byun, Seungone Kim, Kai Shu, and Jinho D. Choi. *Measuring Sycophancy of Language Models in Multi-turn Dialogues.* Findings of EMNLP 2025, pp. 2239–2259. Primary paper.

Primary sources were accessed on October 20, 2026. The related-work discussion describes their task structures, not a new reproduction of their reported results.